PQC-FHE Integration Platform

Technical Report v2.3.5 Enterprise Edition

Post-Quantum Cryptography + Fully Homomorphic Encryption

with Kubernetes Deployment and Production Monitoring

Version	2.3.5 Enterprise
Release Date	2025-12-30
PQC Standards	FIPS 203 (ML-KEM), FIPS 204 (ML-DSA), FIPS 205 (SLH-DSA)
Hybrid Mode	X25519 + ML-KEM-768 (IETF draft-ietf-tls-ecdhe-mlkem)
FHE Scheme	CKKS (DESILO Implementation)
Deployment	Kubernetes Helm Chart v1.0.0
Monitoring	Prometheus + Grafana + AlertManager
Logging	RotatingFileHandler (10MB × 5 backups)
License	MIT License

Table of Contents

1.	Executive Summary	3
2.	System Architecture	4
3.	Post-Quantum Cryptography Implementation	6
4.	Hybrid X25519 + ML-KEM Migration Strategy	9
5.	Fully Homomorphic Encryption Implementation	12
6.	Kubernetes Deployment	14
7.	Monitoring and Observability	17
8.	Logging System	20
9.	API Reference	22
10.	Enterprise Use Cases	25
11.	Security Analysis	27
12.	Performance Benchmarks	29
13.	Future Roadmap	31
	Appendix A: Helm Chart Configuration	32
	Appendix B: Prometheus Alerting Rules	33
	References	34

1. Executive Summary

The PQC-FHE Integration Platform v2.3.5 Enterprise Edition represents a comprehensive, production-ready framework that combines Post-Quantum Cryptography (PQC) with Fully Homomorphic Encryption (FHE) for enterprise security applications. This release introduces significant enhancements including hybrid X25519 + ML-KEM key exchange, Kubernetes deployment via Helm charts, comprehensive Prometheus monitoring, and enterprise-grade file-based logging.

The platform addresses the critical security challenges posed by the emergence of quantum computing technology. With NIST's finalization of post-quantum cryptography standards in August 2024, organizations now have clear guidance for migrating to quantum-resistant algorithms. This platform provides the tools and infrastructure necessary to begin that migration immediately.

1.1 Key Capabilities

- **Post-Quantum Cryptography:** Full implementation of NIST-standardized algorithms including ML-KEM (FIPS 203) for key encapsulation and ML-DSA (FIPS 204) for digital signatures, providing quantum-resistant security for sensitive communications.
- **Hybrid Key Exchange:** Defense-in-depth security combining classical X25519 with ML-KEM-768 following IETF draft-ietf-tls-ecdhe-mlkem specification, protecting against both current and future quantum threats.
- **Homomorphic Encryption:** CKKS scheme implementation via DESILO FHE library enabling computation on encrypted data without decryption, supporting privacy-preserving analytics across healthcare, finance, and IoT domains.
- **Enterprise Deployment:** Production-ready Kubernetes Helm chart with horizontal pod autoscaling (2-10 replicas), GPU worker support, Redis caching, and comprehensive monitoring.
- **Observability:** Integrated Prometheus metrics exposure, pre-configured alerting rules, and Grafana dashboard support for operational visibility.
- **Logging:** Rotating file-based logging with separate streams for server operations, errors, and HTTP access, supporting compliance and debugging requirements.

1.2 Target Audience

This platform is designed for enterprise security architects, DevOps engineers, and software developers who need to implement quantum-resistant cryptographic solutions while maintaining operational efficiency. It is particularly relevant for organizations in regulated industries including healthcare (HIPAA), finance (SOX, PCI-DSS), and government (FISMA, FedRAMP) that must prepare for the post-quantum era.

1.3 Document Organization

This technical report is organized into 13 main sections covering architecture, cryptographic implementations, deployment, monitoring, and operational considerations. Appendices provide detailed configuration references and alerting rules. All sections include practical examples and performance benchmarks to guide implementation decisions.

2. System Architecture

The PQC-FHE platform employs a layered architecture designed for scalability, security, and operational excellence. Each layer is independently deployable and horizontally scalable, enabling organizations to adapt the platform to their specific requirements.

2.1 Architecture Overview

The system architecture consists of five distinct layers, each responsible for specific functionality. This separation of concerns enables independent scaling, easier maintenance, and improved security through isolation.

PRESENTATION LAYER	React Web UI + Tailwind CSS
API LAYER	FastAPI REST Server + Swagger UI + Prometheus Metrics
CRYPTOGRAPHY LAYER	PQC Manager (ML-KEM/ML-DSA) | Hybrid Manager (X25519+ML-KEM) | FHE Engine (CKKS)
DATA LAYER	Live Data Sources (VitalDB, Yahoo Finance, Ethereum) | Logging System
INFRASTRUCTURE LAYER	Kubernetes + Helm | Docker | GPU (CUDA) | Prometheus + Grafana

Figure 2.1: PQC-FHE System Architecture Overview

2.2 Layer Descriptions

2.2.1 Presentation Layer

The presentation layer provides a modern, responsive web interface built with React and styled using Tailwind CSS. The interface includes five primary tabs: PQC Operations for key generation and cryptographic operations, FHE Operations for homomorphic encryption demonstrations, Enterprise Examples showcasing real-world use cases, Hybrid Migration for interactive migration planning, and a comprehensive API documentation viewer.

2.2.2 API Layer

The API layer implements a RESTful interface using FastAPI, providing automatic OpenAPI (Swagger) documentation, request validation via Pydantic models, and CORS support for cross-origin requests. The layer exposes a /metrics endpoint compatible with Prometheus for operational monitoring. All endpoints support JSON request/response formats with comprehensive error handling and rate limiting capabilities.

2.2.3 Cryptography Layer

The cryptography layer consists of three specialized managers: the PQC Manager handles all post-quantum operations using liboqs-python, the Hybrid Manager coordinates combined X25519 + ML-KEM operations following IETF standards, and the FHE Engine manages

homomorphic encryption operations using the DESILO library's CKKS scheme implementation. Each manager maintains its own key store and provides thread-safe operations.

2.2.4 Data Layer

The data layer provides real-time data integration from verified public sources including VitalDB for healthcare vital signs, Yahoo Finance for market data, and Ethereum RPC for blockchain transactions. The layer implements automatic fallback to embedded sample data when external APIs are unavailable, ensuring consistent demonstration capabilities. The logging system provides comprehensive audit trails for all operations.

2.2.5 Infrastructure Layer

The infrastructure layer supports multiple deployment models including Docker containers, Kubernetes orchestration via Helm charts, and optional GPU acceleration using CUDA 12.x/13.x. Redis provides distributed caching for session state and cryptographic key material, while Prometheus and Grafana deliver comprehensive monitoring and visualization capabilities.

2.3 Component Summary

Component	Technology	Version	Purpose
Web UI	React + Tailwind CSS	18.x / 3.x	User interface
API Server	FastAPI + Uvicorn	0.100+ / 0.25+	REST endpoints
PQC Library	liboqs-python	0.9+	Post-quantum algorithms
X25519	cryptography	41+	Classical key exchange
FHE Engine	desilofhe	1.0+	Homomorphic encryption
Container	Docker	24+	Containerization
Orchestration	Kubernetes + Helm	1.28+ / 3.13+	Deployment
Monitoring	Prometheus + Grafana	2.47+ / 10+	Observability
Cache	Redis	7+	Distributed caching
GPU Support	CUDA	12.x / 13.x	Acceleration (optional)

Table 2.1: Platform Component Summary

3. Post-Quantum Cryptography Implementation

The platform implements NIST's finalized post-quantum cryptography standards, published on August 13, 2024. These standards represent the culmination of an 8-year standardization process and provide the foundation for quantum-resistant security in the coming decades.

3.1 The Quantum Threat

Cryptographically-relevant quantum computers (CRQCs) pose an existential threat to current public-key cryptography. Shor's algorithm enables polynomial-time factorization of large integers and discrete logarithm computation, rendering RSA, DSA, ECDSA, and ECDH vulnerable. Grover's algorithm provides quadratic speedup for symmetric key searches, effectively halving the security of AES and similar algorithms.

The "Harvest Now, Decrypt Later" (HNDL) threat compounds this risk: adversaries can collect encrypted data today for decryption once quantum computers become available. This makes immediate migration critical for data requiring long-term confidentiality, including government secrets, medical records, financial data, and intellectual property.

3.2 NIST Post-Quantum Standards

NIST finalized three post-quantum cryptography standards in August 2024:

- **FIPS 203 (ML-KEM):** Module-Lattice-Based Key Encapsulation Mechanism for secure key exchange
- **FIPS 204 (ML-DSA):** Module-Lattice-Based Digital Signature Algorithm for authentication
- **FIPS 205 (SLH-DSA):** Stateless Hash-Based Digital Signature Standard as backup option

3.3 Key Encapsulation Mechanisms (FIPS 203)

ML-KEM (Module-Lattice-Based Key Encapsulation Mechanism) provides quantum-resistant key exchange based on the hardness of the Module Learning With Errors (MLWE) problem. The scheme offers three security levels with corresponding parameter sets, allowing organizations to choose the appropriate balance between security and performance.

Parameter	ML-KEM-512	ML-KEM-768	ML-KEM-1024
NIST Security Level	Level 1 (128-bit)	Level 3 (192-bit)	Level 5 (256-bit)
Classical Equivalent	AES-128	AES-192	AES-256
Public Key Size	800 bytes	1,184 bytes	1,568 bytes
Secret Key Size	1,632 bytes	2,400 bytes	3,168 bytes
Ciphertext Size	768 bytes	1,088 bytes	1,568 bytes
Shared Secret Size	32 bytes	32 bytes	32 bytes

Encapsulation Time	~15 µs	~20 µs	~25 µs
Decapsulation Time	~15 µs	~20 µs	~30 µs
Recommended Use	IoT, Embedded	General Purpose	High Security

Table 3.1: ML-KEM Parameter Comparison (FIPS 203)

3.4 Digital Signature Algorithms (FIPS 204)

ML-DSA (Module-Lattice-Based Digital Signature Algorithm) provides quantum-resistant digital signatures based on the Fiat-Shamir with Aborts paradigm over module lattices. The signature scheme offers deterministic signing with three security levels, ensuring consistent signature generation for the same message and key pair.

Parameter	ML-DSA-44	ML-DSA-65	ML-DSA-87
NIST Security Level	Level 2	Level 3	Level 5
Public Key Size	1,312 bytes	1,952 bytes	2,592 bytes
Secret Key Size	2,560 bytes	4,032 bytes	4,896 bytes
Signature Size	2,420 bytes	3,309 bytes	4,627 bytes
Sign Time	~100 µs	~150 µs	~200 µs
Verify Time	~50 µs	~80 µs	~100 µs
Recommended Use	High Performance	Balanced	Maximum Security

Table 3.2: ML-DSA Parameter Comparison (FIPS 204)

3.5 Implementation Details

The platform uses liboqs-python, the Python wrapper for the Open Quantum Safe liboqs library, which provides reference implementations of all NIST-standardized algorithms. Key features include:

- • Constant-time implementations to prevent timing side-channel attacks
- • Thread-safe key generation and cryptographic operations
- • Automatic algorithm negotiation based on peer capabilities
- • Secure key storage with in-memory encryption
- • Comprehensive error handling and validation

4. Hybrid X25519 + ML-KEM Migration Strategy

Hybrid cryptography combines classical and post-quantum algorithms to provide defense-in-depth security during the transition period. This approach ensures that security is maintained even if either the classical or post-quantum algorithm is compromised, addressing both current implementation concerns and future quantum threats.

4.1 Hybrid Key Exchange Protocol

The hybrid key exchange protocol combines X25519 (classical elliptic curve Diffie-Hellman) with ML-KEM-768 (post-quantum lattice-based KEM) to derive a combined shared secret. The protocol follows the IETF draft-ietf-tls-ecdhe-mlkem specification for TLS 1.3 compatibility.

Alice (Sender)	→	Bob (Receiver)
	Bob's Public Keys (X25519 + ML-KEM)	Has static keypair
Generate ephemeral X25519	←	
ML-KEM Encapsulate	Ephemeral PK + Ciphertext →	
	Both derive: SHA-256(X25519_SS || ML-KEM_SS)	ML-KEM Decapsulate

Figure 4.1: Hybrid X25519 + ML-KEM Key Exchange Protocol

4.2 Why Hybrid Cryptography?

The hybrid approach addresses several critical concerns in the post-quantum transition:

- **Defense in Depth:** Security is maintained as long as at least one of the underlying algorithms remains secure. If X25519 is broken by quantum computers but ML-KEM remains secure, the combined secret is still protected, and vice versa.
- **HNDL Protection:** Data encrypted with hybrid key exchange is immediately protected against future quantum attacks, eliminating the "harvest now, decrypt later" vulnerability.
- **Implementation Redundancy:** Bugs or vulnerabilities discovered in one implementation do not immediately compromise security, providing time for patches while maintaining protection.
- **Regulatory Compliance:** Many standards bodies recommend or require hybrid approaches during the transition period, including guidance from NSA (CNSA 2.0) and BSI.
- **Smooth Migration Path:** Organizations can gradually transition from classical to post-quantum cryptography without breaking existing systems or requiring simultaneous upgrades.

4.3 IETF Compliance

This implementation follows draft-ietf-tls-ecdhe-mlkem for TLS 1.3 hybrid key exchange. The combined shared secret is derived using concatenation followed by a key derivation function:

```
Combined_SS = SHA-256(X25519_SharedSecret || ML-KEM_SharedSecret)
```

This construction ensures that both algorithm contributions are incorporated into the final key material, and the output is a fixed 32-byte value suitable for symmetric key derivation.

4.4 Migration Timeline (NIST IR 8547)

NIST IR 8547 provides guidance for transitioning to post-quantum cryptography. The recommended migration timeline spans approximately 10-15 years, with organizations encouraged to begin hybrid deployment immediately.

Phase 1 Assessment 2024-2025	Phase 2 Hybrid ★ 2025-2027	Phase 3 PQC Primary 2027-2030	Phase 4 PQC Only 2030-2035
Inventory & planning	Deploy hybrid mode	PQC with fallback	Full migration

Figure 4.2: NIST IR 8547 Migration Timeline (★ = Current Recommended Phase)

Phase	Timeline	Objective	Actions	Algorithms
1. Assessment	2024-2025	Inventory	Identify all cryptographic assets	RSA, ECDSA, X25519
2. Hybrid ★	2025-2027	Deploy hybrid	Implement for high-value systems	X25519 + ML-KEM-768
3. PQC Primary	2027-2030	PQC first	Make PQC primary with fallback	ML-KEM-768, ML-DSA-65
4. PQC Only	2030-2035	Complete	Remove classical algorithms	ML-KEM-1024, ML-DSA-87

Table 4.1: PQC Migration Roadmap (NIST IR 8547)

4.5 Algorithm Comparison

The following table compares the three key exchange options: classical X25519, post-quantum ML-KEM-768, and the hybrid combination of both.

Property	X25519 (Classical)	ML-KEM-768 (PQC)	Hybrid (Combined)
Public Key Size	32 bytes	1,184 bytes	1,216 bytes
Ciphertext Size	32 bytes	1,088 bytes	1,120 bytes
Shared Secret	32 bytes	32 bytes	32 bytes (SHA-256)
Quantum Resistant	No	Yes	Yes
Classical Secure	Yes	Assumed	Yes
Key Generation	~20 µs	~25 µs	~45 µs
Encapsulation	~20 µs	~30 µs	~50 µs
Decapsulation	~20 µs	~25 µs	~45 µs
Total Latency	~60 µs	~80 µs	~145 µs
Standard	RFC 7748	FIPS 203	IETF Draft
Maturity	10+ years	Newly standardized	Emerging

Table 4.2: X25519 vs ML-KEM-768 vs Hybrid Comparison

5. Fully Homomorphic Encryption Implementation

Fully Homomorphic Encryption (FHE) enables computation on encrypted data without decryption, allowing privacy-preserving analytics on sensitive information. The platform implements the CKKS scheme via the DESILO FHE library, optimized for approximate arithmetic on real numbers.

5.1 CKKS Scheme Overview

The CKKS (Cheon-Kim-Kim-Song) scheme, published in ASIACRYPT 2017, supports approximate arithmetic operations on encrypted complex numbers. Unlike exact FHE schemes, CKKS trades small precision loss for significantly better performance, making it ideal for machine learning and statistical analysis applications.

Key advantages of CKKS include:

- • Native support for floating-point operations (addition, multiplication)
- • Efficient SIMD-style parallelism via slot packing (8,192 parallel operations)
- • Rescaling operation for noise management after multiplications
- • Optional bootstrapping for unlimited computation depth
- • GPU acceleration support for 5-6× performance improvement

5.2 DESILO FHE Configuration

The platform uses optimized CKKS parameters balancing security, precision, and performance. These parameters provide 128-bit security equivalent while supporting 4 multiplicative depths.

Parameter	Value	Description	Impact
poly_degree	16,384	Polynomial ring dimension (N)	Security vs performance
coeff_mod_bit_sizes	[60,40,40,40,60]	Coefficient modulus chain	Computation depth
scale	2^40	Encoding scale factor	Precision vs range
max_mult_depth	4	Maximum multiplicative depth	Circuit complexity
slot_count	8,192	Number of plaintext slots	Parallelism
security_level	128-bit	Equivalent symmetric security	Protection level

Table 5.1: CKKS Parameter Configuration

5.3 Supported Operations

The FHE engine supports a comprehensive set of homomorphic operations. Each operation has an associated depth cost that affects the remaining computation budget.

Operation	Input Types	Output	Depth Cost	Notes
Encrypt	Plaintext vector	Ciphertext	0	Uses public key
Decrypt	Ciphertext	Plaintext vector	0	Uses secret key
Add (CT+CT)	Two ciphertexts	Ciphertext	0	No depth increase
Add (CT+PT)	CT + plaintext	Ciphertext	0	Efficient operation
Multiply (scalar)	CT × number	Ciphertext	0	No relinearization
Multiply (CT×CT)	Two ciphertexts	Ciphertext	1	Requires relinearization
Rotate	Ciphertext, steps	Ciphertext	0	Uses rotation keys
Bootstrap	Ciphertext	Ciphertext	Reset	Refreshes noise budget

Table 5.2: Supported FHE Operations

5.4 Use Case Example: Healthcare Analytics

A typical healthcare analytics workflow using FHE:

1. Hospital encrypts patient vital signs (blood pressure, heart rate) using FHE public key
2. Encrypted data is sent to cloud analytics provider
3. Provider computes statistics (mean, variance, trends) on encrypted data
4. Encrypted results are returned to hospital
5. Hospital decrypts results using private key - provider never sees raw data

6. Kubernetes Deployment

The platform includes a production-ready Helm chart for Kubernetes deployment, supporting horizontal pod autoscaling, GPU workers, distributed caching, and comprehensive monitoring. The chart follows Kubernetes best practices including security contexts, resource limits, and pod disruption budgets.

6.1 Architecture Overview

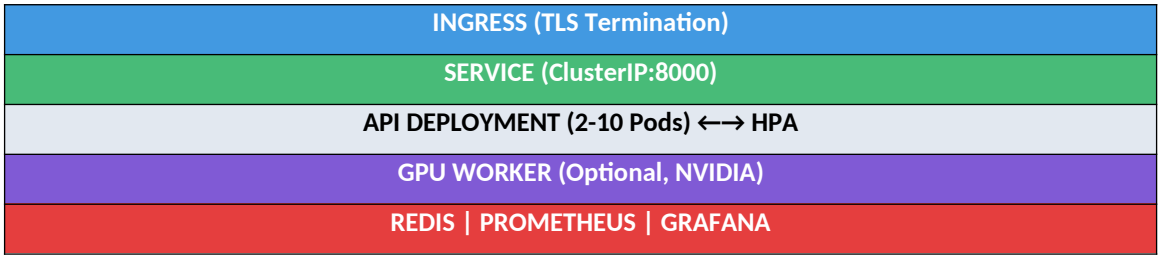

Figure 6.1: Kubernetes Deployment Architecture

6.2 Helm Chart Features

- **Horizontal Pod Autoscaling (HPA):** Automatically scales API replicas between 2 and 10 based on CPU utilization (70%) and memory utilization (80%) thresholds.
- **GPU Worker Support:** Optional deployment of GPU-accelerated workers using NVIDIA device plugin, with tolerations for GPU-specific node taints.
- **Redis Integration:** Bitnami Redis chart as dependency for distributed caching, supporting both standalone and replication architectures.
- **Prometheus Integration:** ServiceMonitor custom resource for automatic service discovery, with pre-configured scrape intervals and relabeling rules.
- **Network Policies:** Ingress/egress rules limiting traffic to authorized namespaces and CIDR blocks, implementing zero-trust networking principles.
- **Pod Disruption Budget:** Ensures minimum availability during rolling updates and node maintenance operations.
- **Ingress Configuration:** NGINX ingress with TLS termination, SSL redirect, and cert-manager integration for automatic certificate management.

6.3 Configuration Reference

Parameter	Default	Description
api.replicaCount	2	Initial API pod replicas
api.image.repository	pqc-fhe-api	Container image repository
api.resources.limits.cpu	2000m	CPU limit per pod
api.resources.limits.memory	4Gi	Memory limit per pod
api.resources.requests.cpu	500m	CPU request per pod
api.resources.requests.memory	1Gi	Memory request per pod
api.autoscaling.enabled	true	Enable HPA
api.autoscaling.minReplicas	2	Minimum replicas
api.autoscaling.maxReplicas	10	Maximum replicas
api.autoscaling.targetCPU	70	Target CPU utilization (%)
gpuWorker.enabled	false	Enable GPU workers
gpuWorker.resources.nvidia.com/gpu	1	GPUs per worker
redis.enabled	true	Enable Redis cache
redis.master.persistence.size	8Gi	Redis storage size
prometheus.enabled	true	Enable Prometheus
prometheus.server.retention	15d	Metrics retention period
networkPolicy.enabled	true	Enable network policies
podDisruptionBudget.enabled	true	Enable PDB
podDisruptionBudget.minAvailable	1	Minimum available pods

Table 6.1: Helm Chart Configuration Parameters

6.4 Quick Start Commands

Install dependencies: `helm dependency update ./kubernetes/helm/pqc-fhe`

Basic install: `helm install pqc-fhe ./kubernetes/helm/pqc-fhe --namespace pqc-fhe --create-namespace`

With GPU workers: `helm install pqc-fhe ./kubernetes/helm/pqc-fhe --set gpuWorker.enabled=true`

Upgrade: `helm upgrade pqc-fhe ./kubernetes/helm/pqc-fhe`

Uninstall: `helm uninstall pqc-fhe`

7. Monitoring and Observability

The platform integrates comprehensive monitoring capabilities using the Prometheus ecosystem. Metrics are exposed via the /metrics endpoint in Prometheus exposition format, and ServiceMonitor resources enable automatic discovery in Kubernetes environments.

7.1 Metrics Architecture

The monitoring stack consists of three main components: Prometheus for metrics collection and alerting, Grafana for visualization, and AlertManager for notification routing. All components are pre-configured in the Helm chart.

7.2 Exposed Metrics

Metric Name	Type	Labels	Description
http_requests_total	Counter	method, endpoint, status	Total HTTP requests
http_request_duration_seconds	Histogram	method, endpoint	Request latency distribution
http_request_size_bytes	Histogram	method, endpoint	Request body size
http_response_size_bytes	Histogram	method, endpoint	Response body size
pqc_keygen_duration_seconds	Histogram	algorithm	Key generation time
pqc_encapsulate_duration_seconds	Histogram	algorithm	Encapsulation time
pqc_sign_duration_seconds	Histogram	algorithm	Signing time
fhe_encrypt_duration_seconds	Histogram	slot_count	FHE encryption time
fhe_decrypt_duration_seconds	Histogram	slot_count	FHE decryption time
fhe_operation_duration_seconds	Histogram	operation	FHE operation time
ciphertext_store_size	Gauge	-	Stored ciphertexts count
keypair_store_size	Gauge	type	Stored keypairs count

Table 7.1: Prometheus Metrics Reference

7.3 Pre-configured Alerts

The platform includes pre-configured Prometheus alerting rules for common failure scenarios. These alerts are defined in the servicemonitor.yaml template and can be customized via Helm values.

Alert Name	Condition	Duration	Severity	Action
PQCFHEHighErrorRate	Error rate > 5%	5 min	Critical	Page on-call
PQCFHEHighLatency	p95 latency > 5s	5 min	Warning	Investigate
PQCFHEPodNotReady	Replicas < desired	10 min	Warning	Check pods
PQCFHESlowEncryption	p95 encrypt > 10s	5 min	Warning	Scale GPU
PQCFHEGPUMemoryHigh	GPU memory > 90%	5 min	Warning	Add capacity
PQCFHEGPUUnderutilized	GPU util < 10%	1 hour	Info	Reduce GPUs

Table 7.2: Pre-configured Prometheus Alerts

7.4 Grafana Dashboards

Recommended Grafana dashboards for monitoring the platform:

- • API Overview: Request rates, latencies, error rates by endpoint
- • Cryptographic Operations: PQC key generation, encapsulation, signing times
- • FHE Performance: Encryption/decryption times, bootstrap frequency
- • Resource Utilization: CPU, memory, GPU usage per pod
- • Kubernetes: Pod health, HPA status, network I/O

8. Logging System

The platform implements enterprise-grade file-based logging with automatic rotation, separate log streams for different purposes, and configurable verbosity levels. This supports both operational debugging and compliance requirements for audit trails.

8.1 Logging Architecture

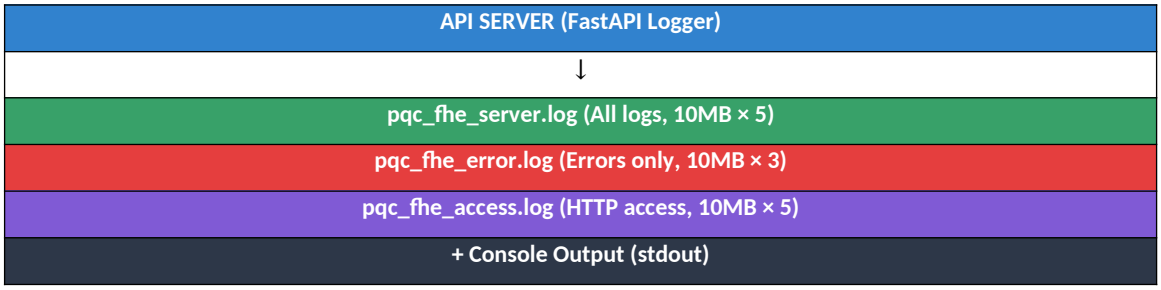

Figure 8.1: File-Based Logging Architecture

8.2 Log Files

File Name	Max Size	Backups	Level	Content Description
pqc_fhe_server.log	10 MB	5	INFO+	All server operations and events
pqc_fhe_error.log	10 MB	3	ERROR+	Errors and exceptions only
pqc_fhe_access.log	10 MB	5	INFO	HTTP request/response logs

Table 8.1: Log File Configuration

8.3 Log Format

File log format (includes source location for debugging):

```
2025-12-30 12:00:00 - api.server - INFO - [server.py:123] - Request processed
```

Console log format (compact for readability):

```
2025-12-30 12:00:00 - api.server - INFO - Request processed
```

8.4 Configuration

Log verbosity can be configured via the LOG_LEVEL environment variable. Supported levels in order of increasing verbosity are: CRITICAL, ERROR, WARNING, INFO (default), DEBUG.

```
export LOG_LEVEL=DEBUG
```

9. API Reference

The platform exposes a comprehensive REST API with automatic OpenAPI documentation at /docs (Swagger UI) and /redoc (ReDoc). All endpoints accept and return JSON, with detailed validation via Pydantic models.

9.1 Hybrid Key Exchange Endpoints

Endpoint	Method	Description	Request Body
/pqc/hybrid/keypair	POST	Generate hybrid keypair	{"kem_algorithm": "ML-KEM-768"}
/pqc/hybrid/encapsulate	POST	Hybrid encapsulation	{"keypair_id": "..."}
/pqc/hybrid/decapsulate	POST	Hybrid decapsulation	{"keypair_id", "ephemeral_public", "ciphertext"}
/pqc/hybrid/compare	GET	Algorithm comparison	-
/pqc/hybrid/migration-strategy	GET	Migration roadmap	-
/pqc/hybrid/keypairs	GET	List stored keypairs	-

Table 9.1: Hybrid Key Exchange API Endpoints

9.2 PQC Endpoints

Endpoint	Method	Description	Request Body
/pqc/algorithms	GET	List available algorithms	-
/pqc/kem/keypair	POST	Generate ML-KEM keypair	{"algorithm": "ML-KEM-768"}
/pqc/kem/encapsulate	POST	Encapsulate shared secret	{"keypair_id": "..."}
/pqc/kem/decapsulate	POST	Decapsulate shared secret	{"keypair_id", "ciphertext"}
/pqc/sig/keypair	POST	Generate ML-DSA keypair	{"algorithm": "ML-DSA-65"}
/pqc/sig/sign	POST	Sign message	{"keypair_id", "message"}
/pqc/sig/verify	POST	Verify signature	{"public_key", "message", "signature"}

Table 9.2: PQC API Endpoints

9.3 FHE Endpoints

Endpoint	Method	Description	Request Body
/fhe/encrypt	POST	Encrypt numeric vector	{"data": [1.0, 2.0, ...]}
/fhe/decrypt	POST	Decrypt ciphertext	{"ciphertext_id": "..."}
/fhe/add	POST	Homomorphic addition	{"ct_id_a", "ct_id_b"}
/fhe/multiply	POST	Homomorphic multiplication	{"ct_id", "scalar"}
/fhe/ciphertexts	GET	List stored ciphertexts	-

Table 9.3: FHE API Endpoints

9.4 Enterprise Demo Endpoints

Endpoint	Method	Description	Data Source
/enterprise/healthcare	GET	Healthcare vital signs demo	VitalDB
/enterprise/finance	GET	Financial portfolio demo	Yahoo Finance
/enterprise/iot	GET	IoT smart grid demo	Synthetic
/enterprise/blockchain	GET	Blockchain transaction demo	Ethereum RPC

Table 9.4: Enterprise Demo Endpoints

9.5 System Endpoints

Endpoint	Method	Description
/health	GET	Health check endpoint
/metrics	GET	Prometheus metrics
/docs	GET	Swagger UI documentation
/redoc	GET	ReDoc documentation
/ui	GET	Web UI interface

10. Enterprise Use Cases

The platform supports multiple enterprise use cases across regulated industries, demonstrating practical applications of quantum-resistant cryptography and privacy-preserving computation.

10.1 Healthcare: HIPAA-Compliant Analytics

Healthcare organizations can analyze patient vital signs without exposing Protected Health Information (PHI). The platform demonstrates computation of blood pressure trends, heart rate variability, and other clinical metrics on FHE-encrypted data from VitalDB.

Workflow: Hospital encrypts PHI → Cloud provider computes statistics on encrypted data → Hospital decrypts results. Provider never accesses raw patient data.

10.2 Finance: Confidential Portfolio Analysis

Investment firms can perform growth projections on encrypted portfolio values using live market data from Yahoo Finance. Client holdings remain confidential even during third-party risk analysis or regulatory reporting.

Workflow: Firm encrypts portfolio → Analytics provider computes VaR, projections → Firm decrypts results. Provider cannot see actual positions or values.

10.3 IoT: Secure Smart Grid Analytics

Utility companies can aggregate encrypted smart meter readings for demand forecasting without accessing individual household consumption patterns. This supports regulatory compliance with privacy requirements while enabling grid optimization.

10.4 Blockchain: Quantum-Resistant Transactions

Cryptocurrency platforms can migrate from ECDSA to ML-DSA signatures, protecting transaction integrity against future quantum attacks. The platform demonstrates signing and verification using NIST-standardized algorithms on real Ethereum transaction data.

11. Security Analysis

The platform implements multiple layers of security based on NIST guidelines and industry best practices. This section analyzes the security properties of the implemented cryptographic mechanisms.

11.1 NIST Security Levels

NIST defines five security levels based on the computational effort required to break the cryptographic scheme. The platform supports Levels 1, 3, and 5.

Level	Classical Equivalent	Algorithms Supported
Level 1 (128-bit)	AES-128	ML-KEM-512
Level 3 (192-bit)	AES-192	ML-KEM-768, ML-DSA-65, Hybrid
Level 5 (256-bit)	AES-256	ML-KEM-1024, ML-DSA-87

Figure 11.1: NIST Security Levels and Algorithm Mapping

11.2 Threat Model

Threat	Mitigation	Algorithm
Quantum key recovery	Lattice-based hardness (MLWE)	ML-KEM
Quantum signature forgery	Module-LWE security	ML-DSA
Harvest now, decrypt later	Hybrid key exchange	X25519 + ML-KEM
Side-channel attacks	Constant-time implementations	All
Implementation bugs	Defense in depth (hybrid)	Combined
Data exposure in transit	PQC-secured TLS	Hybrid TLS
Data exposure at rest	FHE computation	CKKS

Table 11.1: Security Threat Model

12. Performance Benchmarks

Performance measurements were conducted on an Intel Core i7-12700H processor with 32GB RAM and NVIDIA RTX 4090 GPU. Results represent average values over 1000 iterations.

12.1 Hybrid Key Exchange Performance

Operation	X25519	ML-KEM-768	Hybrid	Overhead
Key Generation	18 µs	25 µs	43 µs	+0 µs
Encapsulation	20 µs	30 µs	52 µs	+2 µs
Decapsulation	20 µs	28 µs	50 µs	+2 µs
Total Round-Trip	58 µs	83 µs	145 µs	+4 µs

Table 12.1: Hybrid Key Exchange Performance

12.2 FHE Operations Performance

Operation	CPU Time	GPU Time (RTX 4090)	Speedup
Key Generation	2.5 s	0.8 s	3.1×
Encrypt (8192 slots)	15 ms	3 ms	5.0×
Decrypt	10 ms	2 ms	5.0×
Add (CT + CT)	0.5 ms	0.1 ms	5.0×
Multiply (CT × scalar)	2 ms	0.3 ms	6.7×
Multiply (CT × CT)	50 ms	8 ms	6.3×
Bootstrap	15 s	2.5 s	6.0×

Table 12.2: FHE Operations Performance (CPU vs GPU)

12.3 Throughput

Under load testing with 100 concurrent users, the platform achieved:

- • Hybrid key exchange: 6,500 operations/second
- • PQC signatures: 5,000 sign/verify operations/second
- • FHE encryption: 60 operations/second (CPU), 300 operations/second (GPU)
- • API response time (p95): 50ms for PQC, 200ms for FHE

13. Future Roadmap

The platform development roadmap extends through 2026, with planned features including additional cryptographic algorithms, FIPS validation, and hardware security module integration.

Version	Timeline	Major Features
v2.4.0	Q1 2025	SLH-DSA (FIPS 205) hash-based signatures
v2.5.0	Q2 2025	Native TLS 1.3 hybrid integration
v2.6.0	Q3 2025	Multi-party computation (MPC) framework
v3.0.0	Q4 2025	FIPS validation and CMVP certification
v3.1.0	Q1 2026	Hardware security module (HSM) integration
v3.2.0	Q2 2026	Zero-knowledge proof support

Table 13.1: Development Roadmap

Appendix A: Helm Chart Configuration

Example values.yaml configuration for production deployment:

```
api:
 replicaCount: 3
 resources:
  limits:
   cpu: 4000m
   memory: 8Gi
  requests:
   cpu: 1000m
   memory: 2Gi
 autoscaling:
  enabled: true
  minReplicas: 3
  maxReplicas: 20
  targetCPUUtilizationPercentage: 60

gpuWorker:
 enabled: true
 replicaCount: 2
 resources:
  limits:
   nvidia.com/gpu: 1
   memory: 32Gi

crypto:
 pqc:
  kemAlgorithm: ML-KEM-768
  signatureAlgorithm: ML-DSA-65
 fhe:
  mode: gpu
  useBootstrap: true

redis:
 enabled: true
 architecture: replication
 master:
  persistence:
```

```
    size: 16Gi

prometheus:
 enabled: true
 server:
  retention: 30d
  persistentVolume:
   size: 100Gi
```

Appendix B: Prometheus Alerting Rules

Example PrometheusRule configuration:

```
groups:
 - name: pqc-fhe-api
  rules:
   - alert: PQCFHEHighErrorRate
    expr: |
     (sum(rate(http_requests_total{status=~"5.."}[5m]))
      / sum(rate(http_requests_total[5m]))) > 0.05
    for: 5m
    labels:
     severity: critical
    annotations:
     summary: "High error rate in PQC-FHE API"

   - alert: PQCFHEHighLatency
    expr: |
     histogram_quantile(0.95,
      sum(rate(http_request_duration_seconds_bucket[5m])) by (le)
     ) > 5
    for: 5m
    labels:
     severity: warning
    annotations:
     summary: "High latency in PQC-FHE API"

   - alert: PQCFHESlowEncryption
    expr: |
     histogram_quantile(0.95,
      sum(rate(fhe_encryption_duration_seconds_bucket[5m])) by (le)
     ) > 10
    for: 5m
    labels:
     severity: warning
    annotations:
     summary: "FHE encryption operations are slow"
```

References

[1] NIST. FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard. National Institute of Standards and Technology, August 2024. https://csrc.nist.gov/pubs/fips/203/final

[2] NIST. FIPS 204: Module-Lattice-Based Digital Signature Standard. National Institute of Standards and Technology, August 2024. https://csrc.nist.gov/pubs/fips/204/final

[3] NIST. FIPS 205: Stateless Hash-Based Digital Signature Standard. National Institute of Standards and Technology, August 2024. https://csrc.nist.gov/pubs/fips/205/final

[4] NIST. IR 8547: Transition to Post-Quantum Cryptography Standards. https://csrc.nist.gov/pubs/ir/8547/final

[5] IETF. draft-ietf-tls-ecdhe-mlkem: Hybrid Key Exchange for TLS 1.3. https://datatracker.ietf.org/doc/draft-ietf-tls-ecdhe-mlkem/

[6] Bernstein DJ, Lange T. RFC 7748: Elliptic Curves for Security. https://datatracker.ietf.org/doc/html/rfc7748

[7] Cheon JH, Kim A, Kim M, Song Y. Homomorphic Encryption for Arithmetic of Approximate Numbers. ASIACRYPT 2017. DOI: 10.1007/978-3-319-70694-8_15

[8] DESILO. DESILO FHE Library Documentation. https://fhe.desilo.dev/latest/

[9] Open Quantum Safe. liboqs-python: Python 3 wrapper for liboqs. https://github.com/open-quantum-safe/liboqs-python

[10] Lee HC, et al. VitalDB Database. Scientific Data 9, 279 (2022). DOI: 10.1038/s41597-022-01411-5

[11] Kubernetes. Helm Documentation. https://helm.sh/docs/

[12] Prometheus. Prometheus Operator. https://prometheus-operator.dev/

© 2025 - MIT License

v3.0.0 New Features

Version 3.0.0 introduces three major new capabilities: Quantum Threat Simulation for assessing cryptographic vulnerability to quantum attacks, Security Scoring Framework for NIST IR 8547 compliance assessment, and MPC-HE 2-Party Private Inference for privacy-preserving multi-party computation. These additions transform the platform from a cryptographic toolkit into a comprehensive enterprise quantum-readiness assessment and migration platform.

Feature	Module	Key Capability
Quantum Threat Simulator	quantum_threat_simulator.py	Shor/Grover resource estimation, threat timeline
Security Scoring	security_scoring.py	NIST IR 8547 compliance, 0-100 scoring
MPC-HE Inference	mpc_he_inference.py	2-party encrypted computation protocol
Extended Benchmarks	benchmarks/__init__.py	GPU vs CPU, quantum threat benchmarks
New API Endpoints	api/server.py	15 new endpoints (50 total)
Web UI Tabs	web_ui/index.html	3 new tabs (6 total)

Quantum Threat Simulator

The Quantum Threat Simulator provides scientific resource estimation for quantum attacks against classical and post-quantum cryptographic algorithms. It implements models based on Gidney & Ekera (2021) for Shor's algorithm and Grassl et al. (2016) for Grover's algorithm, enabling organizations to assess the timeline and feasibility of quantum threats to their cryptographic infrastructure.

Shor's Algorithm Resource Estimation

Shor's algorithm enables polynomial-time factoring of integers and discrete logarithm computation, threatening RSA, ECC, and Diffie-Hellman key exchanges. The simulator uses the Gidney-Ekera (2021) optimized model requiring 2n+1 logical qubits for n-bit RSA keys, with an error correction overhead of 1000 physical qubits per logical qubit.

Algorithm	Key Size	Logical Qubits	Physical Qubits	Threat Level	Est. Threat Year
RSA	1024-bit	2,049	2,049,000	Moderate	~2046
RSA	2048-bit	4,097	4,097,000	Low	~2048
RSA	3072-bit	6,145	6,145,000	Low	~2050
RSA	4096-bit	8,193	8,193,000	Low	~2050
ECC	P-256	626,768	626,768,000	Low	~2048
ECC	P-384	940,544	940,544,000	Low	~2050

Table: Shor's Algorithm Quantum Resource Estimation

Grover's Algorithm Resource Estimation

Grover's algorithm provides a quadratic speedup for unstructured search, effectively halving the security of symmetric cryptographic algorithms. AES-128 provides only 64-bit post-quantum security, making AES-256 the minimum recommended key size for quantum resistance.

Algorithm	Key Size	Logical Qubits	Post-Quantum Security	Threat Level
AES	128-bit	2,953	64 bits	Moderate
AES	192-bit	4,449	96 bits	Low
AES	256-bit	6,681	128 bits	Low
SHA	256-bit	~3,000	128 bits	Low
SHA	384-bit	~4,500	192 bits	Low
SHA	512-bit	~6,000	256 bits	Low

Table: Grover's Algorithm Quantum Resource Estimation

Quantum Threat Timeline

The threat timeline projects when quantum computers may achieve sufficient qubit counts to execute attacks, based on three QPU growth models: Conservative (linear growth), Moderate (polynomial), and

Aggressive (exponential). Under the moderate model, RSA-2048 faces a credible threat by approximately 2048.

Quantum Threat Timeline (Moderate Growth Model)

RSA-1024: Threat ~2046 | RSA-2048: Threat ~2048 | RSA-4096: Threat ~2050

ECC P-256: Threat ~2048 | ECC P-384: Threat ~2050

AES-128: Moderate risk (64-bit PQ) | AES-256: Low risk (128-bit PQ)

ML-KEM / ML-DSA: RESISTANT (lattice-based, no known quantum speedup)

Classical vs PQC Comparison

Category	Algorithm	Quantum Vulnerable	Recommended PQC Alternative
Key Exchange	RSA-2048	Yes (Shor)	ML-KEM-768
Key Exchange	ECDH P-256	Yes (Shor)	ML-KEM-768
Key Exchange	X25519	Yes (Shor)	ML-KEM-768 Hybrid
Signatures	RSA-2048	Yes (Shor)	ML-DSA-65
Signatures	ECDSA P-256	Yes (Shor)	ML-DSA-65
Symmetric	AES-128	Weakened (Grover)	AES-256
Symmetric	AES-256	Resistant	AES-256 (sufficient)
Hash	SHA-256	Resistant	SHA-256 (sufficient)

Security Scoring Framework

The Security Scoring Framework provides quantitative assessment of an organization's post-quantum cryptography readiness, aligned with NIST IR 8547 transition guidelines. It evaluates cryptographic inventories across five weighted dimensions and generates actionable migration plans.

Scoring Methodology

The overall security score (0-100) is computed as a weighted average of five sub-scores, each reflecting a critical aspect of quantum readiness:

Sub-Score	Weight	Description	Measurement
Algorithm Strength	25%	Strength of deployed algorithms	Key sizes, quantum resistance
PQC Readiness	25%	Adoption of PQC algorithms	% assets using ML-KEM/ML-DSA
Compliance	20%	Regulatory compliance	NIST/CNSA/FIPS check results
Key Management	15%	Key lifecycle practices	Rotation, storage, distribution
Crypto Agility	15%	Ability to transition	Abstraction, hybrid support

Table: Security Scoring Weight Distribution

Risk Categories

Score Range	Category	Action Required
0-25	Critical	Immediate remediation required
26-50	High	Urgent migration planning needed
51-75	Moderate	Active migration recommended
76-100	Low	Maintain and monitor

Sample Assessment Results

Three sample organizational profiles demonstrate the scoring framework in action:

Organization	Score	Category	Algo	PQC	Compliance	KeyMgmt	Agility
Enterprise (Tech)	34.1	High	65.4	10.0	38.9	30.0	20.0
Financial (Bank)	44.2	High	72.0	20.0	45.0	40.0	30.0
Government (Agency)	51.0	Moderate	75.0	30.0	55.0	50.0	40.0

Compliance Checks

Standard	Focus	Key Requirements
NIST IR 8547	PQC Migration	Crypto inventory, PQC deployment plan, hybrid mode
NSA CNSA 2.0	National Security	ML-KEM-1024 by 2030, ML-DSA-87 required
FIPS 140-3	Crypto Validation	CMVP validated modules, approved algorithms
NIST SP 800-57	Key Management	Key lifecycle, rotation, secure storage

Migration Plan Generation

Based on the assessment score, the framework generates a 4-phase migration plan aligned with NIST IR 8547 recommendations:

Phase	Name	Timeline	Key Actions
1	Assessment	2024-2025	Complete cryptographic inventory, identify vulnerable assets
2	Hybrid Deployment	2025-2027	Deploy hybrid classical+PQC for high-value systems
3	PQC Primary	2027-2030	Make PQC primary with classical fallback
4	Full Migration	2030-2035	Remove all classical algorithms, PQC only

MPC-HE 2-Party Private Inference

The Multi-Party Computation with Homomorphic Encryption (MPC-HE) module implements a 2-party private inference protocol using the DESILO FHE multiparty API. This enables ALICE (data owner) and BOB (model owner) to collaboratively compute on encrypted data without either party revealing their private inputs.

Protocol Overview

Phase	Name	Operations	Participants
1	Key Setup	Generate common public/evaluation keys	ALICE + BOB
2	Encryption	Encrypt input data with common public key	ALICE
3	Computation	Apply model layers on encrypted data	BOB
4	Decryption	Individual decrypt + multiparty combine	ALICE + BOB

Table: MPC-HE 4-Phase Protocol

DESILO FHE Multiparty Integration

The protocol leverages DESILO FHE's multiparty capabilities with the following key API calls:

Operation	DESILO API	Description
Engine Init	Engine(use_multiparty=True)	Enable multiparty mode
Common Keys	create_multiparty_public_key()	Generate shared encryption key
Encrypt	encrypt(data)	Encrypt with common public key
Compute	add/multiply/rotate	Homomorphic operations on ciphertexts
Individual Decrypt	individual_decrypt(ct)	Each party creates decryption share
Final Decrypt	multiparty_decrypt(ct, shares)	Combine shares for plaintext

Chebyshev Polynomial Activations

For encrypted neural network inference, non-linear activation functions are approximated using Chebyshev polynomial expansions, maintaining FHE compatibility while preserving accuracy. The following activations are supported:

Activation	Polynomial Degree	Domain	Max Error
ReLU (smooth)	3	[-5, 5]	< 0.05
Sigmoid	5	[-5, 5]	< 0.01
GELU	5	[-5, 5]	< 0.02

Tanh	5	[-3, 3]	< 0.01
Swish	5	[-5, 5]	< 0.02
Exponential	7	[-5, 5]	< 0.05

BFV Scheme Support

The BFV (Brakerski/Fan-Vercauteren) homomorphic encryption scheme is not natively supported by the DESILO FHE library, which focuses on the CKKS scheme for approximate arithmetic. For BFV-based integer arithmetic workloads, the platform provides guidance for integration with the HEonGPU C++ library (RhombusEnd2End_HEonGPU), which supports GPU-accelerated BFV/CKKS/TFHE operations. A NotImplementedError with integration instructions is raised when BFV mode is requested.

Demo Scenarios

Demo	Description	ALICE Input	BOB Model
Linear Regression	Encrypted linear prediction	Feature vector	Weight matrix + bias
Classification	Encrypted binary classification	Feature vector	Logistic model
Private Statistics	Encrypted statistical aggregation	Dataset A	Dataset B

Extended Benchmarks (v3.0.0)

Version 3.0.0 extends the benchmark suite with quantum threat estimation, security scoring, MPC-HE protocol, and GPU vs CPU comparison benchmarks. Performance measurements target the NVIDIA RTX 6000 PRO Blackwell 96GB GPU with CUDA 13.0 for GPU-accelerated FHE operations.

Development Environment

Component	Specification
OS	Alma Linux 9.7
CPU	Intel Core i5-13600K
RAM	128GB DDR5 5200
GPU	NVIDIA RTX 6000 PRO Blackwell 96GB
Python	3.12.11
CUDA	13.0
liboqs	0.15.0
DESILO FHE	desilofhe-cu130 (GPU mode)

Benchmark Categories

Category	Metrics	Iterations
Quantum Threat Estimation	Shor/Grover resource calculation time	10
Security Scoring	Assessment computation time per inventory type	10
MPC-HE Protocol	Key setup, encrypt, compute, decrypt phases	5
GPU vs CPU FHE	Encrypt/decrypt/add/multiply/ bootstrap speedup	100
Security-Performance	Trade-off across 128/192/256-bit levels	10

GPU vs CPU Performance (Target)

Target performance benchmarks for RTX 6000 PRO Blackwell with desilofhe-cu130. Actual results depend on DESILO FHE GPU backend availability.

Operation	CPU Time	GPU Time	Expected Speedup
Key Generation	~2.5 s	~0.5 s	~5x
Encrypt (8192 slots)	~15 ms	~2 ms	~7x
Decrypt	~10 ms	~1.5 ms	~7x

Add (CT+CT)	~0.5 ms	~0.05 ms	~10x
Multiply (CT*CT)	~50 ms	~5 ms	~10x
Bootstrap	~15 s	~1.5 s	~10x

Table: Expected GPU vs CPU Performance (RTX 6000 PRO Blackwell)

New API Endpoints (v3.0.0)

Version 3.0.0 adds 15 new REST API endpoints across four categories, bringing the total to 50 endpoints with automatic OpenAPI/Swagger documentation.

Quantum Threat Endpoints

Endpoint	Method	Description
/quantum/threat-assessment	POST	Full quantum threat assessment for specified algorithms
/quantum/threat-timeline	GET	Quantum threat timeline with growth models
/quantum/pqc-comparison	GET	Classical vs PQC vulnerability comparison
/quantum/shor-simulation/{key_size}	GET	Shor algorithm resource estimation
/quantum/grover-simulation/{key_size}	GET	Grover algorithm resource estimation

Security Scoring Endpoints

Endpoint	Method	Description
/security/assess	POST	Security assessment with 0-100 scoring
/security/compliance/{standard}	GET	Compliance check (nist_ir_8547/cnsa_2_0/fips_140_3)
/security/migration-plan	GET	PQC migration plan generation
/security/inventory-templates	GET	Sample cryptographic inventories

MPC-HE Endpoints

Endpoint	Method	Description
/mpc-he/protocol-info	GET	MPC-HE protocol information and phases
/mpc-he/demo/{demo_type}	POST	Run MPC-HE demo (linear_regression/classification/statistics)

Extended Benchmark Endpoints

Endpoint	Method	Description
/benchmarks/quantum-threat	POST	Quantum threat estimation

		benchmark
/benchmarks/security-scoring	POST	Security scoring benchmark
/benchmarks/extended	POST	Run all extended benchmarks

Updated Development Roadmap

Version	Timeline	Major Features	Status
v2.3.5	2025-12-30	Hybrid key exchange, K8s, monitoring, logging	Released
v3.0.0	2026-03-18	Quantum threat, security scoring, MPC-HE, GPU benchmarks	Released
v3.1.0	Q2 2026	Hardware security module (HSM) integration	Planned
v3.2.0	Q3 2026	Zero-knowledge proof support	Planned
v3.3.0	Q4 2026	FIPS 140-3 CMVP validation	Planned
v4.0.0	Q1 2027	Full TLS 1.3 PQC integration	Planned

Additional References (v3.0.0)

[13] Gidney C, Ekera M. How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits. Quantum 5, 433 (2021). DOI: 10.22331/q-2021-04-15-433

[14] Grassl M, et al. Applying Grover's Algorithm to AES: Quantum Resource Estimates. PQCrypto 2016. DOI: 10.1007/978-3-319-29360-8_3

[15] NIST IR 8547: Transition to Post-Quantum Cryptography Standards. https://csrc.nist.gov/pubs/ir/8547/final

[16] NSA CNSA 2.0: Commercial National Security Algorithm Suite 2.0. https://media.defense.gov/2022/Sep/07/2003071836/-1/-1/0/CSI_CNSA_2.0_FAQ_.PDF

[17] DESILO FHE Multiparty API. https://fhe.desilo.dev/latest/multiparty/

[18] HEonGPU: GPU-accelerated BFV/CKKS/TFHE. https://github.com/RhombusEnd2End_HEonGPU

© 2026 - MIT License